Java Silver（SE 17・1Z0-825-JPN）　2 Javaの基本データ型と文字列の操作　#03

文字列

String は変わらない、StringBuilder は変わる ― == と equals、連結の順序、主なメソッド、StringBuilder、テキストブロック

各ページの下に、根拠にした Oracle の資料（JLS・API・Text Blocks ガイド）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 2（文字列の作成と操作）

1 String のしくみ
不変、リテラルのプール、== と equals、連結（+）の順序

2 String のメソッド
位置は 0 から。substring は終わりを含まない。戻り値は新しい String

3 StringBuilder
append・insert・delete・replace・reverse。自分自身が変わる

4 テキストブロック
""" と改行で始まる。左の空白と行末の空白は消える。
\
と
\s

	String	StringBuilder
中身	**変わらない**（不変）	**変わる**
変えるメソッド	新しい String を返す	自分を変えて自分を返す
equals	中身を比べる	同じオブジェクトか（== と同じ）
リテラル	"..."・テキストブロック	なし（new で作る）
逆順	reverse なし	reverse()

芯：String のメソッドの戻り値を**受け取らない**コードは、何も変えていない。
本番でいちばん多いひっかけ。

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト（文字列の作成と操作(テキスト・ブロックを含む)）／Java SE 17 API String（Strings are constant; their values cannot be changed after they are created. String buffers support mutable strings）／API StringBuilder（does not override equals）

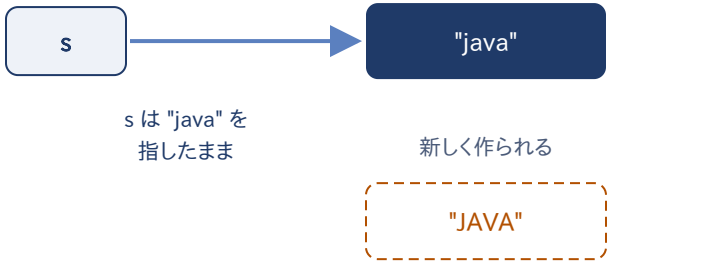
第1章

String のしくみ

変わらない。だから同じリテラルは 1 つを使い回せる。

String は不変 ― 戻り値を受け取らないと変わらない

s.toUpperCase(); のとき（Claude が作成）




```
String s = "java";
s.toUpperCase();          // 戻り値を捨てている
s.concat("!");             // これも
System.out.println(s);    // java

s = s.toUpperCase();      // 受け取れば変わる
System.out.println(s);    // JAVA
```

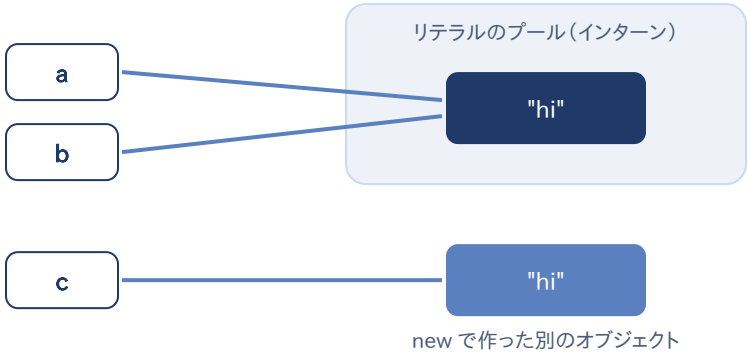
String の「変えるように見える」メソッド（toUpperCase・concat・replace・trim・substring など）は、どれも**新しい String を返す**だけ。

変わるのは変数の中身（参照）だけ：`s = s.toUpperCase();` は、s が新しいオブジェクトを指すようになる。"java" のオブジェクト自体は変わらない。

出典：Java SE 17 API String（Strings are constant; their values cannot be changed after they are created／Because String objects are immutable they can be shared）／結果は JDK 17 で確かめたもの

== と equals、リテラルのプール

オブジェクトの数（Claude が作成）




```
String a = "hi", b = "hi";
String c = new String("hi");
```

式	結果
`a == b`	true（同じオブジェクト）
`a == c`	false（別のオブジェクト）
`a.equals(c)`	true（中身が同じ）
`"Hello" == "Hel" + "lo"`	true（定数式）
`"Hello" == "Hel" + lo`（lo は変数）	false（実行時の連結）
lo が `final String lo = "lo";`	true（定数式になる）
`("Hel" + lo).intern() == "Hello"`	true（プールのものが返る）

中身を比べるなら必ず equals。== は「同じオブジェクトか」。String と StringBuilder を == で比べるのは incomparable types でコンパイルエラー。

出典：JLS 3.10.5 String Literals（a string literal always refers to the same instance of class String … strings that are the values of constant expressions … are "interned"／Example 3.10.5-1）／API String.equals（true if and only if the argument is not null and is a String object that represents the same sequence of characters）／結果は JDK 17 で確かめたもの

連結（+）は左から

1 + 2 + "3" + 4 + 5 の計算（Claude が作成）

1 + 2	数どうし → 足し算 → 3
3 + "3"	片方が String → 連結 → "33"
"33" + 4	→ "334"
"334" + 5	→ **"3345"**

式	結果（JDK 17）
`"" + 'a' + 1`	**"a1"**
`'a' + 1 + ""`	**"98"（char + int は数）**
`s = s + 1 + 2;`（s は "ab"）	**"ab12"**
`t += 1 + 2;`（t は "ab"）	**"ab3"（右辺が先）**
`s += "a";`（s は null）	**"nulla"**
`s.concat("a")`（s は null）	**NullPointerException**
`"x" + true + 2.0`	**"xtrue2.0"**

決まり：+ は**左から**。片方が String なら、もう片方を文字列に変えて**連結**。null は **"null"** になる。+= は右辺を先に計算する。

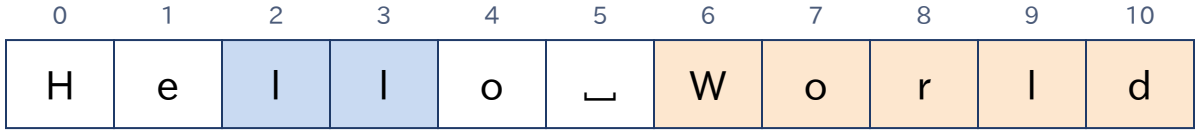
出典：JLS 15.18.1 String Concatenation Operator +（If only one operand expression is of type String, then string conversion is performed on the other operand／The + operator is syntactically left-associative … 1 + 2 + " fiddlers" is: "3 fiddlers"）／JLS 5.1.11 String Conversion（If the reference is null, it is converted to the string "null"）／15.26.2／結果は JDK 17 で確かめたもの

第2章

String のメソッド

位置は 0 から。終わりの位置は含まない。

位置（インデックス）の数え方



substring(b, e) は b 〜 e−1。長さは e − b。e は含まない（delete・StringBuilder.replace も同じ）

式（s は "Hello World"）	結果
`s.charAt(4)`	'o'
`s.charAt(11)`	StringIndexOutOfBoundsException
`s.indexOf('o')`・`s.lastIndexOf('o')`	4・7
`s.indexOf('o', 5)`	7（5 から探す）
`s.indexOf("x")`	-1（ない）

式	結果
`s.substring(11)`	""（空文字列）
`"Hello".substring(3, 2)`	StringIndexOutOfBoundsException
`s.length`・`s.size()`	cannot find symbol
配列 `a.length`・String `s.length()`	かっこの有無に注意

出典：Java SE 17 API String substring（begins at the specified beginIndex and extends to the character at index endIndex - 1. Thus the length of the substring is endIndex-beginIndex）／charAt（IndexOutOfBoundsException - if the index argument is negative or not less than the length of this string）／indexOf（-1 is returned）／結果は JDK 17 で確かめたもの

調べるメソッド・作りかえるメソッド

調べる（例）	結果
`"abcabc".startsWith("bc", 1)`	true（1 から調べる）
`"abcabc".endsWith("")`	true（空はいつも含む）
`"abcabc".contains("ca")`	true
`" ".isEmpty()・" ".isBlank()`	false・true
`"Java".equalsIgnoreCase("JAVA")`	true
`"b".compareTo("a")`	1（後なら正）
`"abc".compareTo("abd")`	-1（前なら負）
`"ab".compareTo("abcd")`	-2（長さの差）
`"Apple".compareTo("apple") < 0`	true（大文字が前）

作りかえる（新しい String を返す）	結果
`"aaa".replace("aa", "b")`	"ba"（左から重ならずに）
`"banana".replace('a', 'o')`	"bonono"（全部）
`" Hi ".trim()`	"Hi"（strip も同じ。全角空白は strip だけ）
`"ab".repeat(3)`	"ababab"
`String.join("-", "a", "b", "c")`	"a-b-c"
`"a,b,,c".split(",").length`	4（途中の空は残る）
`"a,b,c,,".split(",").length`	3（末尾の空は消える）

> **呼ぶたびに新しいオブジェクト：**`s.toUpperCase() == s.toUpperCase()` は false。比べるなら equals。

出典：Java SE 17 API String（startsWith・endsWith（true if the argument is the empty string）・contains・isEmpty・isBlank（empty or contains only white space codepoints）・compareTo・replace・trim・strip・repeat・join・split（Trailing empty strings are therefore not included in the resulting array））／結果は JDK 17 で確かめたもの

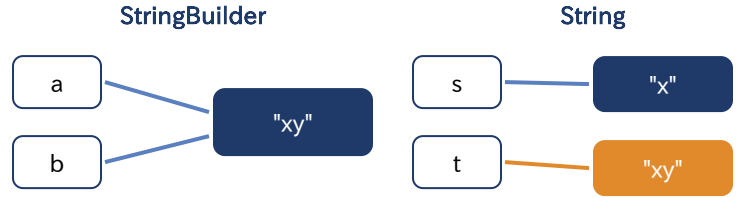
第3章

StringBuilder

同じオブジェクトの中身を変える。だから参照を共有すると全員に見える。

String と StringBuilder のちがい

b = a のあと、それぞれ足すと（Claude が作成）



b.append("y") で 1 つのオブジェクトが変わる → a も "xy"

t = t + "y" は新しい "xy" を作って t に入れる → s は "x"

System.out.println(a + " " + s); → xy x

比べ方	結果（JDK 17）
sb どうしの equals（中身 "ab"）	false（オブジェクトが別）
sb1.toString().equals(sb2.toString())	true
"abc".equals(sb)	false（型がちがう）
"abc".equals(sb.toString())	true
"abc" == sb	コンパイルエラー（incomparable types）

StringBuilder は equals をオーバーライドしていない。中身で比べたいときは toString して String の equals を使う（compareTo でも比べられる）。

容量：new StringBuilder() は 16、new StringBuilder("abc") は 3+16 = 19。足りなければ自動で広がる。length（文字数）とは別。

出典：Java SE 17 API StringBuilder（A mutable sequence of characters／StringBuilder implements Comparable but does not override equals／StringBuilder() … initial capacity of 16 characters／StringBuilder(String str) … initial capacity of 16 plus the length of the string argument）／結果は JDK 17 で確かめたもの

StringBuilder のメソッドを順にたどる

呼び出し	sb の中身
`new StringBuilder("abcdef")`	`abcdef`
`.delete(1, 3)`	`adef`（1〜2 の b・c を消す）
`.insert(0, "X")`	`Xadef`
`.append(1).append('!')`	`Xadef1!`
`.replace(1, 4, "Z")`	`XZf1!`（1〜3 を Z に）
`.deleteCharAt(0)`	`Zf1!`
`.reverse()`	`!1fZ`

上はどれも sb 自身を返すので、つなげて書ける（メソッドチェーン）。表の中身は Claude が手で追った例。

メソッド	注意
append	どんな型も文字にして末尾へ。true は "true"、2.0 は "2.0"
insert(i, x)	i は 0〜length()。こえると StringIndexOutOfBoundsException
delete(b, e)	e は含まない（substring と同じ）
replace(b, e, s)	**位置**で置きかえ。String の replace（**文字列**で探す）と別物
setCharAt・setLength	戻り値は **void**。後ろにつなげられない
toString	今の中身で String を作る

出典：Java SE 17 API StringBuilder（The append method always adds these characters at the end of the builder; the insert method adds the characters at a specified point／delete … extends to the character at index end - 1／replace／reverse／setCharAt・setLength（void））

第4章

テキストブロック

""" と改行で始めて、見たままの複数行を書く。Java 15 から。

テキストブロックの形と、最後の改行

終了の """ を最後の行の末尾に置く

```
String s1 = """
    abc""";           // "abc"（長さ 3）
```

終了の """ を次の行に置く

```
String s2 = """
    abc
    """;              // "abc\n"（長さ 4）
```

NG の例（JDK 17 の表示）

```
String t = """abc""";
// illegal text block open delimiter sequence,
// missing line terminator
String u = "abc
    ";       // unclosed string literal
```

決まり	中身
開始の """	そのあとは空白（任意）と**改行**。同じ行に中身は書けない
中身	開始の改行の**次の行**から、終了の """ の直前まで
最後の改行	""" が単独の行 → 付く。中身の行末 → 付かない
型	**String**。リテラルと同じくインターンされる（"hi\n" と == で true）
" の扱い	そのまま書ける。""" が 3 つ続くときだけエスケープ
メソッド	そのまま呼べる：`"""".formatted("Java")`

出典：JLS 3.10.6 Text Blocks（The opening delimiter … concludes with a line terminator／A text block is always of type String／a text block always refers to the same instance of class String）／Programmer's Guide to Text Blocks（placing the closing delimiter on the last visible line effectively drops the last \n）／結果は JDK 17 で確かめたもの

左の空白はどこまで消えるか

空白を「·」で見えるようにした（Claude が作成）

```
String s = """
··········a
··········b
·····""";
```

灰色の 8 字が消える空白（付随的な空白）。残るのは a の前の 2 字と b の前の 4 字。

s.replace(' ', '.') を表示

```
..a
....b
```

終了の """ の位置	結果（a 10 字・b 12 字下げ）
8 字下げ（中身より左）	`..a` / `....b`
10 字下げ（a と同じ）	`a` / `..b`
14 字下げ（中身より右）	`a` / `..b`（中身の最小に合わせる）
行末に置く（`b"""`）	中身の行だけで決まる。最後の改行なし

決まり：中身の行と「""" だけの最後の行」のうち、**いちばん左の行**に合わせて全体を左に寄せる。行どうしの**相対的な字下げは残る**。

行末の空白も消える。残したいときは `\s` を使う（次のページ）。

出典：JLS 3.10.6（Incidental white space is removed, as if by execution of String.stripIndent）／Programmer's Guide to Text Blocks Incidental White Space（The entire contents of the text block is shifted to the left until the line with the least leading white space has no leading white space）・Trailing White Space／表の結果は JDK 17 で確かめたもの

テキストブロックのエスケープ

```
String s = """
    one \
    two\s
    """;
System.out.println("[" + s.replace("\n", "/") + "]" + s.length());
```

```
[one two /]9
```

行末の `\`:改行を取り消して次の行につなぐ。`\` の前の空白は行末ではないので**残る**。
`\s`:空白 1 つ。エスケープは**空白を消したあと**に変換されるので、行末でも残る。

エスケープ	意味
`\n`・`\t`	改行・タブ（普通の文字列と同じ）
`\"`	"。テキストブロックではふつう不要
`\\`	\ そのもの
`\s`	空白 1 つ（Java 15〜）
`\`+改行	改行しない（テキストブロックの中だけ）

3 つの処理の順番:① 改行を LF にそろえる → ② 左と行末の空白を消す → ③ エスケープを変換する。だから `\s` や `\040` は ② で消されない。

出典:JLS 3.10.6（Line terminators are normalized … Incidental white space is removed … Escape sequences are interpreted）／3.10.7 Escape Sequences（`\s` (space SP, Unicode `\u0020`)／`\` LineTerminator (line continuation)）／Programmer's Guide to Text Blocks（`\s` can act as fence to prevent the stripping of trailing white space）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
String を変えたあとの表示	戻り値を**代入したか**を見る。していなければ元のまま
== の結果	リテラル・定数式どうしは true。new・実行時の連結は false。intern() は true
連結の結果	左から。String が出るまでは数の計算。char + int は数。+= は右辺が先。null は "null"
substring・delete	位置は 0 から、終わりは含まない。長さ ＝ e − b
例外の名前	String・StringBuilder の位置の誤りは **StringIndexOutOfBoundsException**
StringBuilder の結果	sb 自身が変わる。参照をコピーした変数からも見える。equals は同じオブジェクトか
テキストブロックの長さ	最後の改行の有無（""" の位置）、左の空白（いちばん左の行）、\ と \s

出典：このPDFの各ページの資料

まとめ

String のしくみ
不変。同じリテラルは同じオブジェクト。中身は equals で比べる。+ は左から

String のメソッド
0 から数え、終わりは含まない。どれも新しい String を返す

StringBuilder
append・insert・delete・replace・reverse で自分が変わる。equals は == と同じ

テキストブロック
"""+改行で開始。左の空白はいちばん左の行に合わせて消える。
\
と
\s

式	結果
`1 + 2 + "3" + 4 + 5`	"3345"
`"Hello World".substring(2, 4)`	"ll"
`"aaa".replace("aa", "b")`	"ba"
`new SB("abcdef").delete(1, 3)`	"adef"
`"a,b,c,,".split(",").length`	3

次は #04 配列と ArrayList。1 次元・2 次元の配列と、要素数が変わる ArrayList へ進む。

出典：このPDFの各ページの資料（表の SB は StringBuilder の略）